Getting started with the procedures

Overview

The procedure scripts in the `procedures` folder take a device from a configured CRS board to fitted resonances, tuned tones, calibrated timestreams, and photon results. This document explains what all of them have in common: the environment they run in, how they are run, where they save data, and the CRS concepts they use. It also describes `crs_init.py`, which configures the board before any other script runs. The scripts themselves are described in two more documents:

Document	Scripts
VNA screening	`vna_quick_check.py`, `01_vna_sweep.py`, `02_find_resonances.py`, `vna_report.py`
Tuning and timestreams	`03_tune.py`, `04_stream.py`, `05_calibrate_streams.py`, `06_find_photons.py`

The usual order is:

1. `crs_init.py`: configure the board (after a power cycle);
2. `vna_quick_check.py` (optional): check the wiring and the resonances;
3. `01_vna_sweep.py`: full VNA sweep;
4. `02_find_resonances.py`: find and fit the resonances;
5. `vna_report.py` (optional): the VNA results as plain files;
6. `03_tune.py`: find the optimal tone powers and frequencies;
7. `04_stream.py`: take timestreams;
8. `05_calibrate_streams.py`: calibrate the timestreams;
9. `06_find_photons.py`: find photons and cosmic rays.

The environment

The scripts run in citkid's Python environment, which is managed with uv and pinned in `uv.lock`.

1. Install uv.
2. Clone the repository (`git clone https://github.com/loganfoote/citkid`) and go to its folder.
3. Create the environment in `.venv/`, with citkid and the test tools:

   ```
   uv sync
   ```

   The CNN resonance finder used by `02_find_resonances.py` also needs PyTorch: use `uv sync --extra cnn` instead.
4. Activate the environment in each new terminal, so that `python` runs citkid's Python:

   ```
   source .venv/bin/activate      # macOS and Linux
   .venv\Scripts\activate         # Windows
   ```

   `deactivate` leaves the environment.

Instead of activating the environment, a single command can be run in it with `uv run`, from inside the repository:

```
uv run python procedures/01_vna_sweep.py --device deviceA --modules 1
```

From another folder, give the repository's path: `uv run --project /path/to/citkid python script.py`.

Running the scripts

Each script is a short file with a settings block at the top. Copy the script (or edit it in place), set the settings for your setup, and run it. Command-line options override the settings for one run; `--help` lists them. The settings and options of each script are listed in its document.

The device store

Each device has one zarr store per cooldown, `<DATA_DIR>/<COOLDOWN>/<DEVICE>.zarr`, set by the `DATA_DIR`, `COOLDOWN`, and `DEVICE` settings (or `--store` with a path). The store holds all of the data measured and analyzed for the device: VNA sweeps (`vna{k}`), resonator lists (`resonators{k}`), tunings with their calibration sweeps and timestreams (`tune{k}`), stream calibrations (`ts_cal{c}`), and photon results (`ts_photons{p}`). The other two documents describe the groups each script makes.

- Indices are not zero-padded (`vna0`, `vna1`, ...).
- Raw data is never overwritten: repeating a measurement saves it under the next index.
- Every group records when it was created (attribute `created`), and every sweep and stream records when it was taken.
- The store records which CRS it was measured with (serial number, firmware version, and rfmux version). A different board or firmware version needs a new store.
- Calibrated timestreams are never saved. The analysis scripts apply a calibration to the raw streams in memory and save only their results.

To view a store's groups, open it in Python and print its tree:

```
import zarr
print(zarr.open_group('deviceA.zarr', mode = 'r').tree())
```

Notes with the data

Every script that saves data has a `NOTES` setting (or `--notes`). Its text, which can span several lines, is saved with the data in the attribute `notes`. Use it to record, for example, the filters and attenuators on the DAC and ADC lines.

Measuring several devices at once

`crs_init.py` takes global control of the board to configure it. The other scripts only attach to their own modules (`--modules`), so several of them can run at the same time on different devices.

Timestreams are the exception. Streaming uses the board's full network bandwidth, so `04_stream.py` needs exclusive use of the board for as long as its window is open. During that time, any other script that tries to use the board stops immediately with an error that names the running `04_stream.py` session. Run it again after that window is closed. The lock is a file on the DAQ computer (`~/.citkid/locks/crs<serial>.lock`), so it only applies to scripts run on that computer. If a session crashes, its lock is ignored.

Simulated devices

`simulate_store.py` makes a store with a VNA sweep of a simulated device. The scripts that measure (`03_tune.py`, `04_stream.py`) take `--simulate` to measure that device instead of a real one, and the analysis scripts work on the store as on real data. This is useful for learning the procedures and for testing without hardware.

CRS basics

The CRS is the readout board. These are the terms the procedures use.

- **Module**: one readout chain of the board, with its own DAC output (the tones sent to the device) and ADC input (the signal coming back). The board has 8 modules in two analog banks: modules 1-4 and modules 5-8. Only one bank is active at a time; `crs_init.py` chooses it. A device is connected to one module, or to several through a splitter.

- **Tones and channels**: each module can play up to 1024 tones at once, each on its own channel. A tone has a frequency (`fres`) and a power (`ares`, in dBm).
- **DAC full scale**: the largest output power of a module's DAC. The procedures always use 7 dBm; tone powers are set relative to it.
- **NCO**: the numerically controlled oscillator that sets a module's center frequency. A module reads out a band centered on its NCO frequency.
- **Module bandwidth**: the width of that band, 500 MHz, or 600 MHz with extended bandwidth (`crs_init.py --extended-bw`), which has more loss near the band edges. A tone must lie inside the band of the module that plays it. We call a module's band at one NCO frequency an NCO window.
- **Tone sets**: when a device's tones don't all fit in the modules' NCO windows at once, they are divided into tone sets, which are measured one after another with different NCO frequencies (see the tuning and timestreams document).
- **Nyquist zones**: the ADC samples at 5 GHz, so the Nyquist frequency is 2.5 GHz. Frequencies from 100 to 2500 MHz are in the first Nyquist zone, and 2500 to 5000 MHz in the second. A module measures in one zone at a time. Sweeps skip 10 MHz on each side of 2500 MHz, because the point nearest the Nyquist frequency is unreliable.
- **Global control**: changing board-wide settings (clock source, analog bank, bandwidth, decimation stage) needs a session with global control of the board. `crs_init.py` and `04_stream.py` take it; the other scripts only attach to their own modules.

Decimation stages

Streams are averaged down to a lower sample rate in decimation stages 0 (fastest) to 6 (slowest). Sweeps always use stage 6. The sample frequency is 625 MHz / (256 × 64 × 2^stage):

Stage	Sample frequency	Sample period	Tones that can stream at once
0	38.15 kHz	26.2 µs	128, on one module
1	19.07 kHz	52.4 µs	128 per module, on up to 2 modules
2	9.54 kHz	105 µs	1024 on one module, or 128 per module on up to 4
3	4.77 kHz	210 µs	1024 per module on up to 2 modules, or 128 per module on up to 4
4	2.38 kHz	419 µs	1024 per module on up to 4 modules
5	1.19 kHz	839 µs	1024 per module on up to 4 modules
6	596 Hz	1.68 ms	1024 per module on up to 4 modules

The streaming limits come from the network. At fast stages, a module sends its samples in short packets (up to 128 channels) instead of long packets (up to 1024 channels), and only so many packets per second get through. Tones that can't stream at once are streamed in several parts, with the same NCO frequencies.

crs_init.py: configure the board

Run this script before any of the others, and again whenever the board loses its configuration (for example, after a power cycle). It is the only script that changes board-wide settings:

- the clock source;
- the analog bank (modules 1-4, or 5-8);
- the DAC full scale (always 7 dBm);
- the module bandwidth (500 MHz, or 600 MHz with `--extended-bw`);
- the decimation stage used for sweeps (stage 6).

The script exits when it is done, and the board keeps these settings. Running it while other scripts are measuring resets the board in the middle of their measurements.

Settings

```
SERIAL_NUMBER = 46          # CRS serial number
INTERFACE = 'enp5s0'        # Ethernet interface connected to the CRS
```

```
CLOCK_SOURCE = 'VCXO'       # 'VCXO' (internal) or 'SMA' (external 10 MHz)
ANALOG_BANK_HIGH = False    # False: modules 1-4; True: modules 5-8
EXTENDED_BW = False         # True: 600 MHz module bandwidth (more loss
                            # near the band edges)
```

Command line

```
usage: crs_init.py [-h] [--serial SERIAL] [--iface IFACE] [--clock {VCXO,SMA}]
                   [--bank-high] [--extended-bw]

Configure the CRS board.

options:
  -h, --help          show this help message and exit
  --serial SERIAL
  --iface IFACE
  --clock {VCXO,SMA}
  --bank-high
  --extended-bw
```